

Replacing Ad-Hoc Scripts with a Rigorous Cryptanalysis Framework: A Professional Toolkit for POLER Core v4 on Debian 13

System Configuration and Toolchain Management on Debian 13

The foundation of any rigorous cryptanalytic effort is a well-configured and stable computational environment. For the analysis of POLER Core v4, this involves establishing a robust platform on Debian 13 ("trixie") and meticulously managing a specialized Python 3.12 toolchain. The choice of Debian 13 provides access to a mature ecosystem of software packages, including modern versions of essential cryptographic libraries [4](#) [7](#) . However, the specific requirement for Python 3.12 introduces complexities that necessitate a carefully planned approach to environment management and package installation. The primary objective is to create an isolated, reproducible workspace that ensures library compatibility and prevents conflicts with the system's default software stack. This section outlines the necessary steps for system preparation, Python environment isolation, and the strategic installation of the core analytical tools.

A successful implementation begins with addressing the potential discrepancy between the user-specified Python version and the one provided by the official Debian 13 repositories. While Debian 13 may include newer Python versions, historical precedent suggests that the default system interpreter might not always align with cutting-edge releases required by specific scientific libraries [31](#) [32](#) . The recommended practice is to install Python 3.12 as a separate, parallel interpreter. This is typically achieved by downloading the source code from python.org, compiling it, and installing it using the `make altinstall` command. This action places the new interpreter at a location such as `/usr/local/bin/python3.12`, thereby avoiding the replacement of the system-managed `/usr/bin/python3`, which many system utilities depend upon [32](#) . Following this, all subsequent development and analysis work must be conducted within dedicated Python virtual environments. These environments, created using `python3.12 -m venv /path/to/venv`, provide a self-contained directory tree containing a specific Python interpreter and all installed libraries, completely isolating the project from the global `site-packages` directory and other environments [106](#) . This isolation is paramount for

preventing dependency hell, where different projects require conflicting versions of the same library, and for ensuring that the entire analysis pipeline is portable and reproducible 32.

With the isolated Python environment established, the next phase is the systematic installation of the required analytical tools. The Debian 13 system itself provides foundational components through its Advanced Package Tool (APT). Key system-level dependencies, particularly development headers for mathematical and cryptographic libraries, must be installed before attempting to build any software from source. Essential packages include `libgmp-dev` (for GNU Multiple Precision Arithmetic Library), `libmpfr-dev`, `libmpc-dev`, `libflint-dev`, and various compiler toolchains like `build-essential` 199337. Installing these prerequisites upfront significantly reduces the risk of failed compilation attempts later in the process 1. The installation of these packages can be performed with a single command: `sudo apt update && sudo apt install -y <list-of-packages>` 211.

Once the system dependencies are met, the focus shifts to installing the Python-specific libraries, primarily via `pip`. A prioritized installation sequence is recommended to build the analytical capability incrementally. The first priority consists of the three cornerstones of the proposed framework: the Z3 SMT solver, the `gmpy2` arbitrary-precision arithmetic library, and SageMath.

Priority	Tool/Library	Recommended Installation Command(s)	Rationale
CRITICAL	<code>python3-z3</code>	<code>apt install python3-z3</code> 35	Provides the BitVec theory needed to model POLER's $GF(2^{32})$ operations as constraints for finding fixed points and collisions.
CRITICAL	<code>python3-gmpy2</code>	<code>apt install python3-gmpy2</code> 325	Accelerates $GF(2^{32})$ arithmetic by 10-100x, making exhaustive searches and batch computations over the cipher's domain feasible.
CRITICAL	SageMath	<code>pip install sagemath-standard</code> 2	Offers a comprehensive suite of built-in functions for formal algebraic verification, S-box analysis (nonlinearity, LAT/DAT), and Groebner basis computation.

The installation of `python3-z3` and `python3-gmpy2` is straightforward via APT, as pre-compiled packages are available for Debian 35 325. These tools immediately enable the most impactful analyses without requiring complex compilation. `gmpy2` is particularly crucial as it serves as the high-performance engine for all finite field arithmetic, leveraging the underlying GMP C library 93 325. Its availability as a pre-compiled wheel for Python 3.12 further simplifies installation 335.

The installation of SageMath represents the most significant undertaking due to its size and complexity. The `sagemath` package was removed from Debian 13, necessitating alternative installation paths [1](#) . The recommended approach is to use `pip` to install the `sagemath-standard` metapackage. This method downloads and compiles the source code, but it manages the complex dependency graph automatically [2](#) . This process is resource-intensive, requiring several gigabytes of disk space and potentially over an hour of build time on a modern machine [1](#) [2](#) . An alternative is to compile SageMath from source manually, which requires first installing a long list of prerequisite packages found in the Sage documentation before running the `make` command [199](#). This manual method offers more control but carries a higher risk of failure if any dependency is missing. Given the complexity, starting with the `pip install sagemath-standard` command is the most pragmatic first step. It is important to note that SageMath's build process has historically had issues with Python 3.12 due to compiler optimization bugs, so success is not guaranteed and may require patches or waiting for upstream fixes [87](#) .

Following the installation of the core trio, the next tier of tools, offering significant productivity gains, can be added. These include PyNaCl for reference implementations of standard ciphers, and other statistical test suites like NIST STS, Dieharder, and PractRand, which require compilation from source [27](#) [217](#).

Priority	Tool/ Library	Recommended Installation Command(s)	Rationale
HIGH	PyNaCl	<code>apt install python3-nacl</code> 105	Provides reference implementations of modern symmetric ciphers (e.g., ChaCha20-Poly1305) for benchmarking POLER Core v4's performance against industry standards.
HIGH	PARI/GP	<code>apt install pari-gp</code> 202	A powerful computer algebra system focused on number theory, useful for specific algebraic tasks over finite fields.
MEDIUM	TestU01	<code>git clone https://github.com/COMPUTATIONAL-MATH/TestU01.git</code> 123	An industrial-strength battery of statistical tests for random number generators, providing a more extensive evaluation than NIST STS.
MEDIUM	GAP	<code>apt install gap</code> 1	A system for computational discrete algebra, especially group theory, valuable for verifying the quasigroup properties of POLER's operations.
MEDIUM	CryptoSMT	<code>pip install cryptosmt</code> 27	A specialized toolkit for automated differential and linear characteristic search using SAT/SMT solvers, directly applicable to finding weaknesses in the cipher's rounds.

Finally, the statistical test suites—NIST SP 800-22, Dieharder, and PractRand—are essential for evaluating the output randomness of POLER Core v4. As these are not typically packaged for Debian, they must be compiled from their respective source code repositories [89](#) [102](#)[218](#). NIST STS can be obtained from GitHub and compiled with `make`

217. Dieharder may be available in Debian testing repositories 102, but otherwise requires a similar compilation process from source. PractRand is also available on GitHub and compiled with make 90. These tools are designed to read binary data from standard input, making them easy to pipe data from a custom script that generates output from POLER Core v4 88 89.

Throughout this process, careful attention must be paid to the compatibility of each component with Python 3.12. While gmpy2 explicitly supports Python 3.12 96 335, other libraries have had issues. For example, older versions of NumPy were not compatible with Python 3.12 from day one, requiring a new release shortly after the Python version launched 329. Similarly, SageMath's own documentation indicates it targets Python 3.12 as a minimum version 327, but a known bug involving GCC compiler optimizations can cause it to crash when built against this version 87. These potential pitfalls underscore the importance of using isolated virtual environments and being prepared to seek out patched versions or wait for updated releases from upstream developers. By following this structured approach to system configuration and toolchain management, a robust and reliable foundation can be laid for the comprehensive cryptanalysis of POLER Core v4.

Algebraic and Structural Analysis with SageMath

SageMath stands as a central pillar in the proposed research framework, serving as a comprehensive Computational Algebra System (CAS) capable of performing formal, rather than empirical, algebraic analysis of POLER Core v4. Its native support for finite fields, groups, rings, and ideals, combined with a rich set of cryptographic modules, makes it uniquely suited for tasks that would otherwise require extensive, error-prone hand-written code 16 132. The primary application of SageMath in this context is the rigorous verification of the cipher's fundamental algebraic properties, particularly the bijectivity of its core operations, and the detailed evaluation of its substitution box (S-box) components against established cryptographic criteria. This section details the specific methodologies and SageMath functionalities that facilitate this deep structural analysis.

The most critical property of any component in a symmetric cipher is its bijectivity, or permutation nature. For POLER Core v4, which is based on a "deformed tensor product over $GF(2^{32})$ ", it is essential to prove that this operation is a bijection. A statistically

significant sample showing no collisions is insufficient; what is required is a mathematical proof. SageMath provides the tools to achieve this. If the deformed tensor product can be represented as a lookup table mapping elements of $\text{GF}(2^{32})$ to elements of $\text{GF}(2^{32})$, it can be directly fed into SageMath's `sage.crypto.sbox.SBox` class [59](#). This class includes a built-in method, `is_permutation()`, which performs an efficient check to determine if every possible output value is produced exactly once for some input value. A return value of `True` constitutes a definitive proof of bijectivity for the tested domain [59](#) [66](#). This is a far stronger result than a probabilistic test and is a necessary condition for the cipher's round function to be invertible, which is fundamental to symmetric encryption [51](#) [54](#).

If the deformed tensor product is defined by a more complex algebraic expression or a matrix representation over $\text{GF}(2^{32})$, SageMath's broader algebraic capabilities can be leveraged. One can construct the finite field $\text{GF}(2^{32})$ and define the operation's polynomial or matrix form within that field [80](#) [301](#). For instance, if the operation corresponds to multiplication by a matrix M , its bijectivity can be proven by computing its determinant over $\text{GF}(2^{32})$; a non-zero determinant implies the matrix is invertible, and thus the transformation is a bijection [163](#). SageMath excels at these symbolic and numeric computations over finite fields, abstracting away the underlying complexity of the arithmetic [80](#) [114](#). The ability to work with polynomial rings and ideals over finite fields further allows for a deeper investigation into the algebraic structure of the cipher [44](#) [134](#).

Beyond verifying basic properties, SageMath provides a complete suite of tools for the detailed analysis of S-boxes, which are the cornerstone of security in block ciphers. The security of an S-box is typically evaluated based on several key metrics, including nonlinearity, differential uniformity, and adherence to the Strict Avalanche Criterion (SAC) and Bit Independence Criterion (BIC).

Nonlinearity measures the distance of a Boolean function from all affine functions. High nonlinearity is crucial for resisting linear cryptanalysis. For an S-box, its nonlinearity is defined as the minimum nonlinearity of its constituent Boolean component functions. SageMath's `SBox` class computes this value precisely using the Walsh-Hadamard Transform (WHT) [59](#) [60](#). The `nonlinearity()` method internally implements an efficient algorithm for the WHT, often leveraging Fast Fourier Transform (FFT) techniques, to calculate the bias of all possible linear approximations [79](#) [141](#). The result is an exact, quantifiable measure of resistance to linear attacks. For comparison, the AES S-box has a nonlinearity of 112, and achieving values above 104 for an 8-bit S-box is considered excellent [67](#) [68](#). SageMath allows for direct calculation of this metric for POLER's S-box, providing a benchmarkable score.

Differential uniformity assesses the resistance of an S-box to differential cryptanalysis. It is defined as the maximum number of solutions to the equation $S(x \oplus d_1) \oplus S(x) = d_2$ across all non-zero d_1 . An S-box with a low differential uniformity is desirable. A value of 2 corresponds to an Almost Perfect Nonlinear (APN) function, which offers the highest possible resistance [59](#). SageMath's `SBox` class provides the `differential_uniformity()` method, which constructs the full Difference Distribution Table (DDT) and returns the highest entry count [59](#) [60](#). This eliminates the need to manually implement the nested loops required to populate the DDT, reducing both development time and the potential for implementation errors.

SageMath also facilitates the construction of the Linear Approximation Table (LAT) and the Difference Distribution Table (DDT) themselves. The `linear_approximation_table()` method generates the LAT, which is a matrix where each entry (u,v) represents the correlation of the input mask u with the output mask v [59](#). The `difference_distribution_table()` method returns the DDT [59](#). These tables are the raw data for calculating nonlinearity and differential uniformity, but they are also valuable in their own right for visualizing and analyzing the behavior of the S-box. They can be visualized as heatmaps using plotting libraries like matplotlib, allowing for intuitive identification of weak keys or improbable differentials [113](#).

Furthermore, SageMath can be used to analyze the cycle structure of the `polerStep` function, which is another aspect of its dynamical system properties. By iterating the function over the entire domain of $\text{GF}(2^{32})$ and tracking the state transitions, one can decompose the function into a set of cycles. SageMath's `NetworkX` integration is ideal for this task. `NetworkX` is a Python library for the creation, manipulation, and study of the structure, dynamics, and functions of complex networks [131](#). One can use `NetworkX` to build a directed graph where nodes represent states in $\text{GF}(2^{32})$ and edges represent the transition under the `polerStep` function. Once the graph is constructed, algorithms like `nx.cycle_basis()` can be used to find the fundamental cycles, providing insight into the long-term behavior of the cipher's internal state evolution [135](#)[143](#). This analysis, accelerated by `gmpy2` for the $\text{GF}(2^{32})$ operations, can reveal undesirable properties such as short cycles or a large number of small cycles, which could indicate a weakness in the diffusion properties of the cipher.

In summary, SageMath transforms the cryptanalysis of POLER Core v4 from a series of statistical experiments into a discipline grounded in formal mathematics. It provides the means to prove bijectivity, compute exact cryptographic scores for S-box design, and analyze the dynamical properties of the cipher's core components. Its integration with

other tools like `gmpy2` for speed and `NetworkX` for graph theory creates a powerful, synergistic environment for deep structural and algebraic investigation.

Constraint-Based Cryptanalysis Using Z3

While algebraic analysis with SageMath provides a static, structural view of POLER Core v4, constraint-based analysis using an SMT (Satisfiability Modulo Theories) solver like Z3 offers a dynamic, problem-solving approach to uncover exploitable weaknesses. Z3, developed by Microsoft Research, is a highly efficient theorem prover capable of determining the satisfiability of logical formulas with respect to combinations of background theories, including the theory of fixed-size bit-vectors [23, 28]. This capability is exceptionally powerful for cryptanalysis, as it allows the complex logic of a cipher's round function to be encoded as a set of constraints, enabling the solver to either find concrete inputs that satisfy a desired property (like a fixed point) or prove that no such input exists [76, 227]. This methodology replaces slow, brute-force search with sophisticated symbolic execution, dramatically increasing the scale and scope of analyzable problems.

The foundational element for applying Z3 to POLER Core v4 is its support for the theory of bit-vectors (BitVec) [71]. This theory allows for the precise modeling of the bit-level operations that constitute the cipher, such as bitwise XOR ($\wedge$), modular addition, and bit rotations (`rotl`). Each 32-bit word in the cipher's state can be represented as a Z3 BitVec sort of size 32, i.e., `x = BitVec('x', 32)` [29]. The operations of the `polerStep` function, which likely involve these primitives, can then be translated directly into Z3 expressions. For instance, a rotation left by 13 bits, `rotl(x, 13)`, can be expressed using Z3's built-in rotation functions. Modular multiplication over $GF(2^{32})$ can be modeled using Z3's unsigned multiplication and modulo operators. This direct translation is a key advantage, as it maps the cipher's logic onto a formal language that Z3 can reason about efficiently [69].

One of the most direct applications of Z3 in this framework is the search for fixed points. A fixed point is an input `x` such that `polerStep(x) == x`. To find one using Z3, one would first declare a 32-bit bit-vector variable `x`. Then, they would assert the constraint `solver.add(x == polerStep(x))`. The `polerStep(x)` term must be the Z3 expression representing the cipher's round function. After adding this assertion, the solver is queried for a solution with `solver.check()`. If the result is `sat`, a solution is available by calling `solver.model()`, which provides concrete bit assignments for `x` that constitute a fixed point. If the result is `unsat`, it proves that no 32-bit fixed point

exists for the given `polerStep` function, a much stronger statement than failing to find one through random sampling. This technique can be extended to search for cycles of any length by asserting a chain of dependencies (e.g., `y = polerStep(x)`, `z = polerStep(y)`, `x = polerStep(z)` for a 3-cycle).

Similarly, Z3 can be used to search for collisions, which are two distinct inputs `x` and `y` such that `polerStep(x) == polerStep(y)`. This is a critical weakness, as it violates the requirement for the round function to be a permutation. The Z3 model for this problem involves declaring two distinct bit-vector variables, `x` and `y`, and adding the assertion `solver.add(x != y, polerStep(x) == polerStep(y))`. Again, if `solver.check()` returns `sat`, Z3 will produce a model containing a concrete collision pair `(x, y)`. If it returns `unsat`, it proves that the `polerStep` function is injective (and therefore bijective over a finite domain), confirming its permutation property in a formal, non-statistical manner [251](#).

Another powerful application is the automated discovery of structural properties, such as rotational-XOR properties. These are relationships of the form `f(rotl(x, k)) == rotl(f(x), k)`, where `f` is the function being analyzed. Such properties can indicate a lack of sufficient diffusion. Z3 can be used to find such properties by setting up equations that equate the two sides of the relationship and solving for `x` and `k`. For example, one could assert `rotl(polercycle(x), 13) == rotl(polercycle(rotl(x, 13)), 13)` and see if the solver can find any patterns or simplify the expression. This kind of automated structural analysis can reveal hidden symmetries or weaknesses that are difficult to spot manually [184282](#).

The efficiency of Z3's bit-blasting approach, where bit-vectors are translated into propositional logic, is crucial for handling the complexity of a full cipher round [117](#). However, this approach has limitations, particularly with non-linear integer arithmetic, where it may fall back to less efficient methods or even return 'unknown' when dealing with mixed theories [120332](#). Despite this, for bit-precise operations common in ciphers, Z3 is exceptionally effective [28](#). The integration of Z3 with Python via the `python3-z3` package makes it accessible for scripting and automation, allowing analysts to programmatically generate constraints, solve them, and parse the results. The combination of Z3 with `gmpy2` is also beneficial; while Z3 handles the logical constraints, `gmpy2` can be used in the surrounding Python script to perform high-speed arithmetic for generating test vectors or for defining sub-components of the `polerStep` function that are too complex to encode directly in Z3's BitVec theory [93](#).

In essence, Z3 elevates the cryptanalysis of POLER Core v4 from a descriptive science to a prescriptive one. Instead of just measuring properties, it actively searches for violations of those properties. By transforming the cipher's logic into a formal problem, it enables the automated discovery of fixed points, collisions, and other exploitable structures, providing a powerful complement to the algebraic verification performed in SageMath.

Statistical Validation and Randomness Testing of Cipher Output

After verifying the algebraic structure and searching for structural weaknesses, the final stage of the preliminary analysis focuses on the most fundamental goal of any secure cipher: producing output that is indistinguishable from random noise. This is assessed through a battery of standardized statistical tests designed to detect any non-random behavior, such as biases in bit frequency, correlations between bits, or predictable patterns. For POLER Core v4, this involves generating a sufficiently long stream of ciphertext (e.g., by encrypting a constant plaintext block with a sequence of different keys) and feeding this binary data into established statistical test suites. The primary tools for this task on the Debian 13 platform are the NIST Statistical Test Suite (STS), Dieharder, and PractRand. Each suite offers a unique collection of tests, and their collective use provides a comprehensive evaluation of the cipher's pseudorandom properties.

The National Institute of Standards and Technology (NIST) Special Publication 800-22 is the gold standard for statistical testing of random and pseudorandom number generators intended for cryptographic applications [218221](#). It provides a battery of 15 distinct tests, each designed to detect a different kind of non-randomness. Key among these are the Frequency (Monobit) test, which checks if the proportion of zeros and ones is approximately the same; the Runs test, which examines the total number of runs of ones and zeros of various lengths; and the Serial test, which analyzes the frequency of all possible pairs of bits [216220](#). Other important tests include the Binary Matrix Rank test, which constructs matrices from the bitstream and checks their rank; the Discrete Fourier Transform (Spectral) test, which looks for periodic characteristics; and tests for approximate entropy, cumulative sums, and random excursions [216217](#). The NIST STS is available for download from public repositories, such as GitHub, and is compiled from source [217](#). To use it, one typically generates a large file of binary output from POLER Core v4 and pipes it to the `stat_tests.py` script, which invokes the appropriate C

executables for each test. A passing result for a test is indicated by a p-value greater than a significance level (typically 0.01), and a majority of the tests should pass for the generator to be considered statistically random [222](#).

While NIST STS is comprehensive, other test suites offer additional rigor and longer test sequences. Dieharder is an extensive collection of over 30 statistical tests derived from George Marsaglia's earlier Diehard tests, plus additional tests contributed by the community [102](#). It includes tests for overlapping permutations, parking lot calculations, and the squeeze test, which can be more sensitive to certain types of biases than those in the NIST suite [102208](#). On Debian, Dieharder may be available directly via APT (`apt install dieharder`), or it may need to be compiled from source [102](#). Like NIST STS, it expects binary input, and its output provides a detailed pass/fail report for each individual test, allowing for fine-grained analysis of weaknesses [224](#).

PractRand represents a significant advancement in statistical testing by pushing the limits of testable data volume. It is capable of processing terabytes of data (up to 32 TB in some configurations), which allows it to detect subtle biases that only become apparent over very long sequences [91](#) [190](#). PractRand is designed to interface with external RNGs by reading their output from standard input, making it trivially easy to connect it to a script that generates data from POLER Core v4 [88](#) [89](#). The tool is highly optimized for memory usage, enabling it to maintain complex state information about the generated sequence without consuming excessive resources [190](#). A typical usage pattern involves a command like `your_poler_script | RNG_test stdin` [89](#). PractRand is particularly adept at detecting failures in the equidistribution properties of the generator and can often identify looping or short-period behavior that other suites might miss [292](#). Its ability to test packed binary data is a key feature, ensuring that the input format is correct for meaningful analysis [322](#).

The installation of these statistical test suites typically requires compiling from source. The general procedure is to clone the respective Git repository and follow the included README or Makefile instructions. For NIST STS, this involves cloning the repository and running `make` [217](#). For PractRand, the process is similar: `git clone https://github.com/richgel999/practrand.git` followed by `make` [90](#). It is crucial that the binary data provided to these testers is in the correct format. Most expect raw binary data (bytes) on standard input or in a file, not ASCII-encoded hexadecimal or text representations of bits [322](#). The endianness of the system must also be considered when interpreting the byte stream, though for most statistical tests, as long as the data is consistent, the specific endianness of the host system is not a concern [206254](#).

The results from these statistical tests provide a critical reality check on the effectiveness of POLER Core v4's design. A failure in this phase does not necessarily imply a catastrophic vulnerability that allows for key recovery, but it indicates a failure to achieve the basic principles of confusion and diffusion. For example, if the cipher exhibits poor avalanche properties, it will likely fail SAC tests, which is a precursor to susceptibility to linear and differential cryptanalysis. Therefore, a cipher that fails even a single statistical test is considered insecure for cryptographic purposes. Passing these batteries of tests does not guarantee security, as they are not designed to find structural weaknesses like fixed points or collisions. However, failing them is a definitive sign of a flawed design. The combination of NIST STS for a standardized baseline, Dieharder for additional sensitivity, and PractRand for long-range behavior provides a powerful and comprehensive validation of the cipher's output quality.

Advanced Analytical Techniques and Performance Optimization

Beyond the foundational algebraic, structural, and statistical analyses, the proposed framework incorporates advanced techniques and performance-enhancing tools to push the boundaries of the cryptanalytic investigation. This section explores two key areas: the application of Gröbner basis attacks for algebraic cryptanalysis, facilitated by SageMath, and the critical role of high-performance libraries like `gmpy2` in making these computationally intensive analyses feasible. These advanced methods, while more complex and demanding on computational resources, offer the potential to uncover vulnerabilities that are invisible to simpler forms of analysis.

Gröbner basis computation is a powerful technique from computational algebra that can be applied to solve systems of multivariate polynomial equations, a common problem in algebraic cryptanalysis [272](#). The strategy involves converting the description of a cipher (e.g., its S-boxes and round function) into a system of polynomial equations over a finite field, such as $\text{GF}(2)$ or $\text{GF}(2^{32})$. If a cipher has a simple algebraic structure, it may be possible to solve this system to recover the secret key or find other weaknesses. SageMath provides a robust and user-friendly interface for this type of analysis through its `sage.crypto.sbox.SBox` class [59](#). The `polynomials()` method of an `SBox` object can generate a set of multivariate polynomials that describe the S-box's behavior [60](#) [66](#). When called with the argument `groebner=True`, SageMath computes a reduced Gröbner basis for this system of polynomials [60](#). A Gröbner basis has the property that

the remainder of any polynomial upon division by the basis is unique, and it can be used to determine if a solution exists and to find it. For instance, if the system of equations can be simplified to a triangular form via a Gröbner basis, it becomes much easier to solve for the variables step-by-step [244](#).

However, the practical application of Gröbner basis attacks on a cipher like POLER Core v4 faces significant challenges. The computation of Gröbner bases is notoriously complex, with a worst-case complexity that is doubly exponential in the number of variables [47](#). Even for moderately sized systems, the computation can take hours or days and may consume vast amounts of memory [41](#). The efficiency of the computation depends heavily on the choice of monomial ordering (e.g., lexicographic, graded reverse lexicographic) and the specific structure of the polynomial system. While SageMath uses highly optimized underlying engines like FGb and Singular, the feasibility of a successful attack is not guaranteed [46](#) [48](#). Therefore, this technique should be considered a "best-effort" analysis. It is most promising if POLER Core v4 is suspected of having a simple, low-degree algebraic representation. The output from this analysis can provide definitive proof of a cipher's resistance to algebraic attacks—if a Gröbner basis for the system can be computed quickly, it suggests the system is complex enough to be secure. Conversely, a rapid computation yielding a simple triangular system would be a major red flag. Parallelization techniques and improvements in algorithms continue to make these computations more tractable, but they remain a frontier of cryptanalytic research [196](#).

Parallel to the theoretical power of algebraic techniques lies the practical necessity of computational speed. The analysis of POLER Core v4, particularly tasks involving exhaustive search over its 32-bit state space (which has 2^{32} , or over 4 billion, elements), is inherently computationally expensive. Pure Python's built-in `int` arithmetic, while versatile, is not optimized for the large integer and modular arithmetic required for finite field operations and is consequently orders of magnitude slower than specialized C-based libraries [325](#). This is where `gmpy2` becomes an indispensable tool. `gmpy2` is a Python extension module that wraps the highly optimized GNU Multiple Precision Arithmetic Library (GMP) [325](#). It provides fast multiple-precision integer, real, and complex arithmetic, with a C-coded backend that delivers performance comparable to hand-tuned C code [325](#)[328](#).

The impact of `gmpy2` on performance is dramatic. For batch operations over $GF(2^{32})$ —such as populating a difference distribution table (DDT) by iterating through all possible input differences, or analyzing the cycle structure of the `polerStep` function—it can reduce computation times from hours to minutes [325](#). For example, constructing a 256x256 DDT requires $256 \times 256 = 65,536$ evaluations of the S-box. Performing this in a

loop thousands of times to gather statistics would be excruciatingly slow with pure Python integers. With `gmpy2`, these modular multiplications, additions, and bit manipulations are executed at near-C speeds [93](#). The library provides direct support for essential number-theoretic functions like modular inversion (`invert(a, m)`) and fast modular exponentiation (`powmod(a, b, m)`), which are fundamental building blocks for finite field arithmetic [93](#).

Integrating `gmpy2` into the analysis workflow is straightforward. Anywhere a calculation over $GF(2^{32})$ is performed, integers can be cast to `gmpy2.mpz` objects. For example, instead of `result = (a <em> b + c) % mod`, one would use `from gmpy2 import mpz; a, b, c, mod = map(mpz, [a, b, c, mod]); result = (a </em> b + c) % mod`. This single change can yield massive performance benefits across an entire script. Furthermore, `gmpy2`'s ability to handle arbitrarily large integers is also beneficial when working with SageMath, as it can serve as the underlying arithmetic engine for many of Sage's finite field constructions, ensuring consistency and speed [325](#). The availability of pre-compiled wheels for `gmpy2` on PyPI that target Python 3.12 further simplifies its integration into the project's virtual environment [335](#).

In conclusion, the combination of advanced algebraic techniques like Gröbner basis computation with the raw performance provided by `gmpy2` creates a formidable analytical capability. While Gröbner basis attacks are a long shot and computationally intensive, they represent a potential path to discovering deep structural flaws. In parallel, `gmpy2` acts as the engine that makes large-scale, brute-force-style analyses of the cipher's properties a practical reality. Together, they elevate the cryptanalysis from a qualitative assessment to a quantitative, computationally-driven investigation of the cipher's resilience.

Integrated Research Workflow and Implementation Strategy

The culmination of this analysis is the formulation of an integrated, multi-stage research workflow that synthesizes the individual tools and techniques into a coherent and systematic process for verifying the security of POLER Core v4. This workflow is designed to progress logically from foundational algebraic properties to detailed component analysis, structural integrity, and finally, statistical validation. It provides a clear, actionable roadmap for the researcher, ensuring that each phase builds upon the last and

that the entire process is guided by professional-grade, battle-tested instruments available on the Debian 13 platform. The strategy emphasizes reproducibility, efficiency, and the replacement of ad-hoc scripts with a robust, automated framework.

The first phase of the workflow is **Foundational Algebraic Verification**, centered on using SageMath to establish the cipher's most basic mathematical properties. 1.

Environment Setup: Begin by creating a dedicated virtual environment with Python 3.12 (`python3.12 -m venv poler_env`) and activating it. Install the critical dependencies: `python3-z3`, `python3-gmpy2`, and `sagemath-standard` via `pip` [2](#) [32](#) [35](#) [325](#). Also install supplementary libraries like `numpy`, `scipy`, and `matplotlib` for future data analysis and visualization [56](#). 2. **Bijectivity Proof:** Obtain the specification of POLER's deformed tensor product operation. Implement this operation in Python, ensuring all arithmetic is performed using `gmpy2.mpz` for correctness over $\text{GF}(2^{32})$. Create a lookup table of size 2^{32} if possible, or define the operation as a function. Use this to instantiate SageMath's `SBox` object. Call the `is_permutation()` method to obtain a definitive, mathematically proven result on the operation's bijectivity [59](#) [66](#). This step replaces any previous statistical sampling with a formal verification.

The second phase, **Component-Level Security Analysis**, focuses on evaluating the cryptographic strength of the S-box derived from the tensor product. 1. **S-Box**

Construction: Extract the S-box values from the cipher's design documents. Feed these values into the SageMath `SBox` constructor. 2. **Quantitative Assessment:** Execute a suite of analytical methods on the `SBox` object:

- Compute ``sbox.nonlinearity()`` to get a score for resistance to linear cryptanalysis [59](#).
- Compute ``sbox.differential_uniformity()`` to assess resistance to differential cryptanalysis [59](#).
- Run ``sbox.sac()`` and ``sbox.bic()`` to check compliance with the Strict Avalanche and Bit Independence Criteria [59](#).

1. **Data Collection:** Capture the numerical outputs of these methods. These values form a quantitative scorecard for the S-box, which can be compared against benchmarks from established ciphers like AES [67](#).

The third phase, **Constraint-Based Structural Integrity**, employs Z3 to search for exploitable weaknesses in the round function. 1. **Modeling the Round Function:**

Carefully translate the `polerStep` function from its original language (e.g., Zig or Python) into a Z3-compatible representation. Each 32-bit register should be a Z3 `BitVec` of size 32. All logical and arithmetic operations must be mapped to their Z3 equivalents.

2. **Fixed Point Search:** Create a Z3 solver instance. Declare a 32-bit variable x . Add the assertion $x == \text{polerStep}(x)$. Check for satisfiability. If `sat` is returned, extract the model to find a concrete fixed point. If `unsat` is returned, a formal proof of the absence of 32-bit fixed points is achieved [76](#). 3. **Collision Search:** Repeat the process for collisions. Declare two distinct variables x and y . Assert $x \neq y$ and $\text{polerStep}(x) == \text{polerStep}(y)$. A satisfying model reveals a collision, while an `unsat` result proves the function is a permutation [227](#).

The fourth and final phase is **Statistical Validation of Output Quality**. 1. **Output Generation:** Write a script to generate a large binary file of output from POLER Core v4. A common method is to encrypt a fixed plaintext block (e.g., all zeros) repeatedly, using a counter as the key. Generate at least 1 GB, and preferably 10 GB or more, of data for thorough testing. 2. **Run Statistical Tests:** Pipe this binary data to the various statistical test suites:

- **NIST STS:** Run the generated file through the NIST Statistical Test Suite to obtain a standardized pass/fail report against 15 key tests [217](#).
- **Dieharder:** Use Dieharder for a more extensive set of tests that can provide additional sensitivity to subtle biases [102](#).
- **PractRand:** Employ PractRand to test the longest possible sequence and detect any long-range correlations or periodicities that other suites might miss [91](#) [188](#).

1. **Analysis of Results:** Compile the pass/fail rates from all tests. A single failure across the suites is a strong indicator of a design flaw.

This integrated workflow transforms the cryptanalysis of POLER Core v4 from a fragmented collection of disparate scripts into a systematic, rigorous, and reproducible research program. It leverages the unique strengths of each tool: SageMath for formal algebra, Z3 for constraint solving, and statistical suites for validating randomness. By following this structured approach, the researcher can comprehensively address the security claims of the cipher, moving beyond superficial checks to a deep, multi-faceted investigation of its properties, thereby fulfilling the core research goal with professional-grade precision.

Reference

1. Sagemath no longer available in Debian 13 - Reddit https://www.reddit.com/r/debian/comments/1q3uhwk/sagemath_no_longer_available_in_debian_13/
2. Package sage/sage-debian-trixie-standard-with-targets - GitHub <https://github.com/sagemath/sage/pkgs/container/sage%2Fsage-debian-trixie-standard-with-targets>
3. Updated Debian 13: 13.5 released <https://www.debian.org/News/2026/20260516>
4. Debian 13 Trixie has been released – DietPi Blog <https://dietpi.com/blog/?p=4014>
5. Installation Guide - SageMath Documentation <https://doc.sagemath.org/html/en/installation/index.html>
6. Debian 13.4 “Trixie” Released with 111 Bug Fixes and 67 Security ... <https://9to5linux.com/debian-13-4-trixie-released-with-111-bug-fixes-and-67-security-updates>
7. Debian 13 "Trixie" Complete Guide: Installation, Security & Server ... <https://dev.to/dargslan/debian-13-trixie-complete-guide-installation-security-server-setup-2026-3jeb>
8. Installation — Cryptography 50.0.0-dev1 documentation <https://cryptography.io/en/latest/installation/>
9. The Debian 13 (Trixie) Landscape - Broadcom TechDocs <https://techdocs.broadcom.com/us/en/ca-enterprise-software/layer7-api-management/api-gateway/11-2/install-configure-upgrade/upgrade-the-gateway/-the-debian-13--trixie--landscape.html>
10. Debian 13.5 point release lands with security fixes, bug patches <https://www.helpnetsecurity.com/2026/05/18/debian-13-5-released/>
11. [PDF] Exploring Cryptography Using the Sage Computer Algebra System <https://www.sagemath.org/files/thesis/nguyen-thesis-2009.pdf>
12. Classical Cryptosystems - Cryptography - SageMath Documentation <https://doc.sagemath.org/html/en/reference/cryptography/sage/crypto/classical.html>
13. Rijndael-GF - Cryptography https://doc.sagemath.org/html/en/reference/cryptography/sage/crypto/mq/rijndael_gf.html
14. SageMath and Python Code - CrypTool <https://www.cryptool.org/en/ctbook/sagemath/>
15. [PDF] Crash Course: SageMath for Post-Quantum Cryptography https://yx7.cc/docs/sage/sagepqc_taipei_slides.pdf

16. [PDF] Sage for Lattice-based Cryptography <https://www.maths.ox.ac.uk/system/files/attachments/sage-introduction.pdf>
17. [PDF] Exploring Lattices and Algebraic Structures in SageMath | CyBOK https://www.cybok.org/wp-content/uploads/Exploring_Lattices_and_Algebraic_Structures_in_SageMath.pdf
18. [PDF] Cryptography - SageMath Documentation <https://doc.sagemath.org/pdf/en/reference/cryptography/cryptography.pdf>
19. Cryptanalysis and Improvements of the Quasigroup Block Cipher <https://www.semanticscholar.org/paper/179f849bdf51cca61508b176377ba1a1f9ada3b7>
20. SageMath: Deeper Insights into Cybersecurity - Open Source For You <https://www.opensourceforu.com/2025/04/sagemath-deeper-insights-into-cybersecurity/>
21. [PDF] Comparative Analysis of SMT Solvers for Differential Cryptanalysis ... https://ceur-ws.org/Vol-4008/SMT_paper18.pdf
22. [PDF] Differential cryptanalysis with SAT, SMT, MILP, and CP <https://eprint.iacr.org/2024/105.pdf>
23. Science And Engineering - Fun With Flags - Z3 SMT Solver <https://sites.google.com/site/janbeck/cybersecurity-and-reverse-engineering-fun/fun-with-flags-z3-smt-solver>
24. [PDF] Introduction to SMT with Z3 | ISEC - TU Graz <https://www.isec.tugraz.at/wp-content/uploads/2019/09/Introduction-to-SMT-with-Z3.pdf>
25. Z3 - Microsoft Research <https://www.microsoft.com/en-us/research/project/z3-3/>
26. [PDF] Z3: An Efficient SMT Solver - ALGOS <http://algos.inesc-id.pt/projectos/timewarp/ref13.pdf>
27. GitHub - kste/cryptosmt: An easy to use tool for cryptanalysis of ... <https://github.com/kste/cryptosmt>
28. (PDF) Z3: an efficient SMT solver - ResearchGate https://www.researchgate.net/publication/225142568_Z3_an_efficient_SMT_solver
29. [PDF] SMT Solver Z3 <https://ece.uwaterloo.ca/~agurfink/stqam.w20/assets/pdf/W04-UsingZ3.pdf>
30. Z3 Explained - Satisfiability Modulo Theories & SMT Solvers <https://www.youtube.com/watch?v=EacYNe7moSs>
31. Will Debian Trixie shipped with python 3.12 - Reddit https://www.reddit.com/r/debian/comments/1k0mvy3/will_debian_trixie_shipped_with_python_312/
32. How to Install Python 3.12 on Debian 13, 12 and 11 - LinuxCapable <https://linuxcapable.com/how-to-install-python-3-12-on-debian-linux/>
33. LNbts fails to start on Debian 13 (Trixie) build due to Python 3.13 ... <https://github.com/raspiblitz/raspiblitz/issues/5205>

34. Installing ComfyUI with Python 3.12 on Debian 13 (Trixie) with CUDA <https://dadhacks.org/2025/12/05/installing-comfyui-with-python-3-12-on-debian-13-trixie-with-cuda/>
35. Debian -- Details of package python3-z3 in sid <https://packages.debian.org/sid/python3-z3>
36. Need Python3.12 - Technical Support - Toradex Community <https://community.toradex.com/t/need-python3-12/28168>
37. Debian Contributions: Preparing for Python 3.12, /usr-merge ... <https://www.freexian.com/blog/debian-contributions-10-2023/>
38. Compatibility with Python-3.13 - Google Groups <https://groups.google.com/g/eden-asp/c/70mUwjLwYfl>
39. Update Python3 to Debian Stable | LINUXMAKER <https://www.linuxmaker.com/en/how-to/update-python3-to-debian-stable.html>
40. Python 3.13 backport for Debian 12 bookworm - Community Guides <https://community.home-assistant.io/t/python-3-13-backport-for-debian-12-bookworm/842333>
41. Groebner basis computation extremely slow - Ask SageMath <https://ask.sagemath.org/question/83270/groebner-basis-computation-extremely-slow/>
42. Educational version of the (d) -Groebner basis algorithm over PIDs https://doc.sagemath.org/html/en/reference/polynomial_rings/sage/rings/polynomial/toy_d_basis.html
43. Debian -- Package Search Results -- sagemath <https://packages.debian.org/sagemath>
44. Ideals in multivariate polynomial rings - SageMath Documentation https://doc.sagemath.org/html/en/reference/polynomial_rings/sage/rings/polynomial/multi_polynomial_ideal.html
45. Groebner Fans - Combinatorial and Discrete Geometry https://doc.sagemath.org/html/en/reference/discrete_geometry/sage/rings/polynomial/groebner_fan.html
46. Singular and Sage Grobner bases [closed] - Math Stack Exchange <https://math.stackexchange.com/questions/4705251/singular-and-sage-grobner-bases>
47. How to speed up the process for calculating the Groebner basis? <https://mathoverflow.net/questions/433072/how-to-speed-up-the-process-for-calculating-the-groebner-basis>
48. A Sage interface for FGb (Gröbner bases) - Google Groups <https://groups.google.com/g/sage-devel/c/cxs2fNKecLo>
49. ACVP Symmetric Block Cipher Algorithm JSON Specification <https://pages.nist.gov/ACVP/draft-celi-acvp-symmetric.html>

50. RFC 8645 - Re-keying Mechanisms for Symmetric Keys <https://datatracker.ietf.org/doc/rfc8645/>
51. [PDF] Symmetric Encryption - ByDesign https://bydesign-project.eu/wp-content/uploads/guidance-documentation/_static/Appendix_8a.pdf
52. [PDF] Chapter 4 Symmetric Encryption <https://cr.yp.to/bib/2004/bellare-chap4.pdf>
53. Symmetric Algorithms - CompTIA Security+ SY0-501 - 6.2 <https://www.professormesser.com/security-plus/sy0-501/symmetric-algorithms/>
54. [PDF] Popa & Wagner Spring 2016 CS 161 Computer Security 2/22 ... <https://inst.eecs.berkeley.edu/~cs161/archive/sp16/notes/2.22.symmetric.pdf>
55. [PDF] Symmetric Encryption (Block Ciphers) <https://homes.luddy.indiana.edu/yh33/Teaching/I433-2016/lec4-symcrypt-blockciphers.pdf>
56. Symmetric-Key Cryptography | Computer Security <https://textbook.cs161.org/crypto/symmetric.html>
57. aes - Do all forms of symmetric encryption require multiple pieces of ... <https://crypto.stackexchange.com/questions/81212/do-all-forms-of-symmetric-encryption-require-multiple-pieces-of-information-no>
58. [PDF] CRYSTALS-Kyber Algorithm Specifications And Supporting ... <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210131.pdf>
59. S-Boxes and Their Algebraic Representations - Cryptography <https://doc.sagemath.org/html/en/reference/cryptography/sage/crypto/sbox.html>
60. `sage/src/sage/crypto/sbox.pyx` at develop · sagemath/sage · GitHub <https://github.com/sagemath/sage/blob/develop/src/sage/crypto/sbox.pyx>
61. S-Boxes used in cryptographic schemes - Cryptography <https://doc.sagemath.org/html/en/reference/cryptography/sage/crypto/sboxes.html>
62. Constructing 8×8 S-Boxes with Optimal Boolean Function ... - MDPI <https://www.mdpi.com/2410-387X/9/4/67>
63. [PDF] A 10-bit S-box generated by Feistel construction from cellular ... <https://eprint.iacr.org/2025/457.pdf>
64. [PDF] ALGORITHM FOR GENERATION OF S-BOX USING ... <https://ijctcm.researchcommons.org/cgi/viewcontent.cgi?article=1473&context=journal>
65. A highly nonlinear substitution-box (S-box) design using action of ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC7660566/>
66. S-Boxes and Their Algebraic Representations <http://sporadic.stanford.edu/reference/cryptography/sage/crypto/sbox.html>
67. how to calculate non linearity of AES S-box? <https://crypto.stackexchange.com/questions/19968/how-to-calculate-non-linearity-of-aes-s-box>

68. [PDF] Optimizing Nonlinear Substitutions in Symmetric Ciphers - arXiv <https://arxiv.org/pdf/2407.03510>
69. [PDF] Understanding Bit-vector Arithmetic in Z3 - TU Delft Repository https://repository.tudelft.nl/file/File_0517332f-750c-464e-ad27-0a144d8f672f
70. Programming Z3 - Stanford CS Theory <https://theory.stanford.edu/~nikolaj/programmingz3.html>
71. Bitvectors | Online Z3 Guide <https://microsoft.github.io/z3guide/docs/theories/Bitvectors/>
72. Z3Guide: A Scalable, Student-Centered, and Extensible Educational ... <https://arxiv.org/html/2506.08294v1>
73. How to model signed integer with BitVector? - Stack Overflow <https://stackoverflow.com/questions/17838424/how-to-model-signed-integer-with-bitvector>
74. z3py: setting model completion seems to have no effect #1920 <https://github.com/Z3Prover/z3/issues/1920>
75. Shallow Embedding Logics in Z3 pt. I - Philip Zucker https://www.philipzucker.com/shallow_logic_knuckle/
76. Some Silly Z3 Scripts I Wrote - Hillel Wayne <https://www.hillelwayne.com/post/z3-examples/>
77. Boolean functions - Cryptography https://doc.sagemath.org/html/en/reference/cryptography/sage/crypto/boolean_function.html
78. Symbolic Computation - Symbolic Calculus <https://doc.sagemath.org/html/en/reference/calculus/sage/calculus/calculus.html>
79. sage/src/sage/crypto/boolean_function.pyx at develop - GitHub https://github.com/sagemath/sage/blob/develop/src/sage/crypto/boolean_function.pyx
80. Symbolic calculations in finite field extension - Ask SageMath <https://ask.sagemath.org/question/34330/symbolic-calculations-in-finite-field-extension/>
81. 2016-04-27: Symbolic Calculus using SageMath (part 2 of 3) <https://www.youtube.com/watch?v=g8DI9eRgnBw>
82. Symbolic Computation — Sage 9.3.beta9 Reference Manual <http://sporadic.stanford.edu/reference/calculus/sage/calculus/calculus.html>
83. [PDF] An Introductory Course on Sage https://www.math.uni-potsdam.de/fileadmin/user_upload/An_Introductory_Course_on_Sage.pdf
84. [PDF] Symbolic tensor calculus on manifolds: a SageMath implementation <https://people-lux.obspm.fr/gourgoulhon/pdf/GourgM18.pdf>
85. [PDF] Implementing Hadamard Matrices in SageMath - arXiv <https://arxiv.org/pdf/2306.16812>

86. A Brief Introduction to Sage Math <https://melczer.ca/files/SageIntro.html>
87. the sagemath saga <https://trofi.github.io/posts/312-the-sagemath-saga.html>
88. Tests_overview.txt - PractRand https://practrand.sourceforge.net/Tests_overview.txt
89. manual page for practrand-RNG_test using PractRand version 0.95 https://www.mankier.com/1/practrand-RNG_test
90. randomgen/tools/practrand-driver.py at main - GitHub <https://github.com/bashtage/randomgen/blob/master/tools/practrand-driver.py>
91. PCG Passes PractRand | PCG, A Better Random Number Generator <http://www.pcg-random.org/posts/pcg-passes-practrand.html>
92. Sage Input Formatting - Utilities - SageMath Documentation https://doc.sagemath.org/html/en/reference/misc/sage/misc/sage_input.html
93. mpz type - gmpy2's documentation! <https://gmpy2.readthedocs.io/en/latest/mpz.html>
94. PractRand (Practically Random) RNG testing - John D. Cook <https://www.johndcook.com/blog/2020/02/05/using-practrand-to-test-rng/>
95. How to build an exact MPFR number using gmpy2? - Stack Overflow <https://stackoverflow.com/questions/79612338/how-to-build-an-exact-mpfr-number-using-gmpy2>
96. gmpy2 - PyPI <https://pypi.org/project/gmpy2/2.2.2a1/>
97. Debian 13 Trixie Released - Reddit https://www.reddit.com/r/debian/comments/1mlwjz5/debian_13_trixie_released/
98. News -- Debian 13 "trixie" released <https://www.debian.org/News/2025/20250809>
99. Debian 13 (Trixie) Review – What's New, Installation & Upgrade ... <https://www.youtube.com/watch?v=11eQ6J8WNBQ>
100. Lucky 13: a look at Debian trixie - LWN.net <https://lwn.net/Articles/1033474/>
101. Debian 13 (Trixie) — What You Need to Know - Nubius Solutions <https://nubius.io/debian-13-trixie-what-you-need-to-know/>
102. Versions for dieharder - Repology <https://repology.org/project/dieharder/versions>
103. Linux: Debian 13 comes as Trixie | heise online <https://www.heise.de/en/news/Linux-Debian-13-comes-as-Trixie-10515473.html>
104. Debian 13 Trixie Release Date is Officially Confirmed! - OSTechNix <https://ostechnix.com/debian-13-trixie-release-date-confirmed/>
105. Please add support for Debian 13 Trixie · Issue #1056 · boltgolt/howdy <https://github.com/boltgolt/howdy/issues/1056>
106. Installing Python packages in Debian Trixie - KevCaz's Website https://kevcaz.insileco.io/notes/2023/08/2023-08-10_installing_python_packages/

107. [PDF] Symmetric block ciphers with a block length of 32 bit https://opus.hsfurtwangen.de/files/4714/Thesis_v01.pdf
108. [PDF] Automatic differential cryptanalysis of symmetric ciphers - PORTO@Iris https://iris.polito.it/retrieve/71ab9e32-1306-4c60-b3fc-307f820bd399/conv_thesis_rossi_final.pdf
109. [PDF] Side Channel Cryptanalysis of Product Ciphers - SciSpace <https://scispace.com/pdf/side-channel-cryptanalysis-of-product-ciphers-4q9rkx7b1c.pdf>
110. Scalable Neural Cryptanalysis of Block Ciphers in Federated Attack ... <https://www.mdpi.com/2227-7390/14/2/373>
111. [PDF] Cryptanalysis of Some Lightweight Symmetric Ciphers - DTU Inside https://backend.orbit.dtu.dk/ws/files/52690277/orbit_thesis.pdf
112. [PDF] Analysis and Design of Block Cipher Constructions https://informatik.rub.de/wp-content/uploads/2021/11/thesis_andrey.pdf
113. [PDF] STATISTICAL CRYPTANALYSIS OF BLOCK CIPHERS <https://infoscience.epfl.ch/server/api/core/bitstreams/df9c2b85-468e-4b4d-ae42-31fe9c1bca44/content>
114. [PDF] Algebraic methods in block cipher cryptanalysis - SageMath <https://www.sagemath.org/files/thesis/weinmann-thesis-2009.pdf>
115. [PDF] Bachelor's Thesis in Computing Science https://www.cs.ru.nl/bachelors-theses/2023/Els_de_Haan___1012212___Differential_Cryptanalysis_of_the_SIMON_block_cipher.pdf
116. Cryptanalysis of symmetric encryption algorithms - ResearchGate https://www.researchgate.net/publication/331009568_Cryptanalysis_of_symmetric_encryption_algorithms
117. Z3 Internals (Draft) <https://z3prover.github.io/papers/z3internals.html>
118. [PDF] Satisfiability Modulo Theories: A Beginner's Tutorial - Haniel Barbosa <https://hanielbarbosa.com/papers/fm2024.pdf>
119. [PDF] INTRODUCTION TO SMT - Ilya Sergey https://ilyasergey.net/CS5232/_static/slides/08-Intro-to-SMT.pdf
120. Mixing theories in SMT - Stack Overflow <https://stackoverflow.com/questions/37159731/mixing-theories-in-smt>
121. WORKSHOP: SAT/SMT Solvers - YouTube <https://www.youtube.com/watch?v=DX3G4IoTNf0>
122. goldenscale/GS_GithubMirror: A Github Mirror List For GS Star ... https://github.com/goldenscale/GS_GithubMirror

123. [PDF] TestU01 and Practrand: Tools for a randomness evaluation for ... - HAL <https://hal.science/hal-02993846v1/document>
124. PRNG performance with TestU01 or PractRand · Issue #8 - GitHub <https://github.com/vnmakarov/mum-hash/issues/8>
125. Changing the Bitstream File Format Settings - 2026.1 English - UG908 <https://docs.amd.com/r/en-US/ug908-vivado-programming-debugging/Changing-the-Bitstream-File-Format-Settings>
126. 2. What's new in Debian 13 — release-notes documentation <https://www.debian.org/releases/stable/release-notes/whats-new.html>
127. vc2-bitstream-validator - BBC Open Source https://bbc.github.io/vc2_conformance/cli/vc2_bitstream_validator.html
128. <https://ci.debian.net/packages/p/publican/testing/i386> <https://ci.debian.net/packages/p/publican/testing/i386>
129. Debian -- Package Search Results -- jupyter <https://packages.debian.org/jupyter>
130. Debian -- Software Packages in "forky", Subsection math <https://packages.debian.org/testing/math/>
131. [PDF] Groups - SageMath Documentation <https://doc.sagemath.org/pdf/en/reference/groups/groups.pdf>
132. [PDF] Tutorial - SageMath Documentation https://doc.sagemath.org/pdf/en/tutorial/sage_tutorial.pdf
133. [PDF] General Rings, Ideals, and Morphisms - SageMath Documentation <https://doc.sagemath.org/pdf/en/reference/rings/rings.pdf>
134. [PDF] Noncommutative Polynomials - SageMath Documentation https://doc.sagemath.org/pdf/en/reference/noncommutative_polynomial_rings/noncommutative_polynomial_rings.pdf
135. Dynamical systems on projective schemes - SageMath Documentation https://doc.sagemath.org/html/en/reference/dynamics/sage/dynamics/arithmetric_dynamics/projective_ds.html
136. [PDF] Introduction to Symbolic Computation <http://homepages.math.uic.edu/~jan/mcs320/introductionToSymbolicComputation.pdf>
137. hadipourh/autoguess - GitHub <https://github.com/hadipourh/autoguess>
138. [PDF] Sage Tutorial in Russian - SageMath Documentation https://doc.sagemath.org/pdf/ru/tutorial/SageTutorial_ru.pdf
139. [PDF] arXiv:2309.11290v1 [math.NT] 20 Sep 2023 <https://arxiv.org/pdf/2309.11290>
140. [PDF] Syllabus for M.Sc. (Mathematics) <https://www.sssihl.edu.in/wp-content/uploads/2022/04/MSc-Mathematics-Syllabus-from-2020-21-Batch-onwards.pdf>

141. [PDF] Faster Pseudorandom Correlation Generators via Walsh-Hadamard ... <https://eprint.iacr.org/2026/196.pdf>
142. A Dynamical Reformulation of the Hodge Conjecture via Mass ... <https://www.facebook.com/groups/38680135622/posts/10172521374955623/>
143. [PDF] Strongly regular graphs - CWI <https://homepages.cwi.nl/~aeb/math/srg/rk3/srgw.pdf>
144. pg - GitHub Gist <https://gist.github.com/kipply/3c145c544e9fc46d543e53e6b5e06c7c>
145. AGI-Edgerunners/LLM-Agents-Papers - GitHub <https://github.com/AGI-Edgerunners/LLM-Agents-Papers>
146. NeurIPS 2025 Papers with Code & Data - Paper Digest <https://www.paperdigest.org/2025/11/neurips-2025-papers-with-code-data/>
147. RSA cryptosystem - Wikipedia https://en.wikipedia.org/wiki/RSA_cryptosystem
148. IMCG/awesome-c - GitHub <https://github.com/IMCG/awesome-c>
149. Mathematical Markup Language (MathML) Version 4.0 - W3C <https://www.w3.org/TR/mathml4/>
150. PMD Core Ontology <https://materialdigital.github.io/core-ontology/>
151. GPT Image 2 Prompt Gallery + Agentic Skill + CLI - GitHub <https://github.com/wuyoscar/GPT-Image2-Skill>
152. Loss of TJP1 disrupts gastrulation patterning and increases ... - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC10529434/>
153. [PDF] Cryptanalysis, Reverse-Engineering and Design of Symmetric ... <https://orbilu.uni.lu/bitstream/10993/31195/1/thesis.pdf>
154. [PDF] Survey of Computational Assumptions Used in Cryptography Broken ... <https://crypto.cs.mcgill.ca/~crepeau/PDF/memoire-hong.pdf>
155. [PDF] International Congress of Mathematicians <https://www.mathunion.org/fileadmin/IMU/ICM2010/offline/www.icm2010.in/wp-content/icmfiles/abstracts/Contributed-Abstracts-5July2010.pdf>
156. Trudy Matematicheskogo Instituta imeni V.A. Steklova: Archive https://www.mathnet.ru/php/archive.phtml?jrnid=tm&wshow=contents&option_lang=eng&viewarchiveID=12
157. [PDF] ANNUAL REPORT - Erwin Schrödinger Institute https://www.esi.ac.at/static/assets/extras/reports/ESI_Report_2019.479ae92f4d23.pdf
158. Trudy Mat. Inst. Steklova, Author Index - Math-Net.Ru https://math-net.ru/php/contents.phtml?jrnid=tm&wshow=aindex&option_lang=eng
159. [PDF] of the American Mathematical Society <https://community.ams.org/journals/notices/201001/201001-full-issue.pdf>

160. [PDF] A Complete Bibliography of Publications in Bulletin canadien de ... <https://ftp.math.utah.edu/pub/tex/bib/canmathbull.pdf>
161. [PDF] ICMASE 2025 25 - Abstract Book https://icmase.com/uploadfiles/files/2025_abstract_book.pdf
162. The Princeton Companion to Applied Mathematics 9781400874477 <https://dokumen.pub/the-princeton-companion-to-applied-mathematics-9781400874477.html>
163. [PDF] libro de resúmenes - Biental RSME 2026 https://2026.bientalrsme.com/sites/default/files/Libro_resumenes_RSME_2026.pdf
164. Design and optimization of nonlinear component of block cipher <https://www.sciencedirect.com/science/article/pii/S2090447923003969>
165. [PDF] Systematically Quantifying Cryptanalytic Non-Linearities in Strong ... <https://eprint.iacr.org/2022/1766.pdf>
166. BoolSPLG: A Library with Parallel Algorithms for Boolean Functions ... <https://www.mdpi.com/2227-7390/11/8/1864>
167. [PDF] Design, Implementation, and Analysis of a Block Cipher Based on a ... <https://hal.science/hal-03799417v1/file/applsci-12-09952%20%282%29.pdf>
168. Quantum cryptanalysis of SPN ciphers with known plaintext - Nature <https://www.nature.com/articles/s41534-026-01218-x>
169. Catalog and Illustrative Examples of Lightweight Cryptographic ... https://link.springer.com/chapter/10.1007/978-3-030-10591-4_2
170. [PDF] New Directions in Garbled Circuits A Dissertation Presented to The ... <https://repository.gatech.edu/server/api/core/bitstreams/bba9d340-effe-41e9-a865-50354653871d/content>
171. Guide to FFT Analysis (Fast Fourier Transform) | Dewesoft <https://dewesoft.com/blog/guide-to-fft-analysis>
172. Optical tweezers — from calibration to applications: a tutorial <https://opg.optica.org/aop/abstract.cfm?uri=aop-13-1-74>
173. Details of package python3.13 in trixie <https://packages.debian.org/trixie/python3.13>
174. Broken Dependencies on Debian trixie #4487 - Xpra-org/xpra - GitHub <https://github.com/Xpra-org/xpra/issues/4487>
175. Update Debian 12 to 13: Trixie | Raspberry Pi / Debian Guide https://www.youtube.com/watch?v=P_bGZHkTP7Y
176. Projected Tensor-Tensor Products for Efficient Computation ... - arXiv <https://arxiv.org/html/2409.19402v1>
177. [PDF] Scalable Higher-Order Tensor Product Spline Models <https://proceedings.mlr.press/v238/ruegamer24a/ruegamer24a.pdf>

178. Z3 bitvector unsatisfiable after adding more than one constraint <https://stackoverflow.com/questions/79052614/z3-bitvector-unsatisfiable-after-adding-more-than-one-constraint>
179. Enabling Efficient Equivariant Operations in the Fourier Basis via... <https://openreview.net/forum?id=mhyQXJ6JsK>
180. Performance issue when solving bit-vectors #6395 - GitHub <https://github.com/Z3Prover/z3/issues/6395>
181. [PDF] Towards a Geometric Understanding of Tensor Learning via the t ... https://proceedings.neurips.cc/paper_files/paper/2025/file/cef8f771a55c5ff0c9a421813c7f598b-Paper-Conference.pdf
182. Tensors for Beginners 13: Tensor Product vs Kronecker Product https://www.youtube.com/watch?v=qp_zg_TD0qE
183. Comparison between the best models from the tensor product and ... https://www.researchgate.net/figure/Comparison-between-the-best-models-from-the-tensor-product-and-the-irreducible-tensor_fig3_393526333
184. [PDF] Differential cryptanalysis of "An optimized novel lightweight block ... <https://eprint.iacr.org/2025/2142.pdf>
185. [PDF] Cryptanalysis of Selected Block Ciphers - DTU Research Database https://orbit.dtu.dk/files/123428642/phd360_Alkhzaimi_HA.pdf
186. [PDF] Cryptanalysis of Block Ciphers: A Survey <https://perso.uclouvain.be/fstandae/PUBLIS/U1.pdf>
187. [PDF] Strong Keys for Tensor Isomorphism Cryptography - DROPS <https://drops.dagstuhl.de/storage/00lipics/lipics-vol345-mfcs2025/LIPIcs.MFCS.2025.78/LIPIcs.MFCS.2025.78.pdf>
188. How to Test with PractRand - PCG Random Number Generation <http://www.pcg-random.org/posts/how-to-test-with-practrand.html>
189. x264(1) — x264 — Debian testing - Debian Manpages <https://manpages.debian.org/testing/x264/x264.1>
190. What is the development status of PractRand - SourceForge <https://sourceforge.net/p/pracrand/discussion/366935/thread/53eefb08/>
191. <https://openqa.debian.net/tests/489979> <https://openqa.debian.net/tests/489979>
192. Passing various RNG Tests - Cryptography Stack Exchange <https://crypto.stackexchange.com/questions/119739/passing-various-rng-tests>
193. Hyperplane Arrangements - Combinatorial and Discrete Geometry https://doc.sagemath.org/html/en/reference/discrete_geometry/sage/geometry/hyperplane_arrangement/arrangement.html

194. [PDF] Packages and Features - SageMath Documentation <https://doc.sagemath.org/pdf/en/reference/spkg/spkg.pdf>
195. [PDF] ACA 2015 - SINGACOM <http://www.singacom.uva.es/ACA2015/latex/ACAproc.pdf>
196. ECO Seminar - LIRMM <https://www.lirmm.fr/eco/seminar.php>
197. [PDF] The CrypTool Book: Learning and Experiencing Cryptography with ... <https://www.csd.uoc.gr/~hy458/2022/resources/files/CrypTool-Book-en-v12-May18.pdf>
198. [PDF] On Low-Rank Multiplicity-Free Fusion Categories <https://mural.maynoothuniversity.ie/id/eprint/18958/1/phd.pdf>
199. How to install SageMath in Ubuntu in 24.04? <https://askubuntu.com/questions/1516582/how-to-install-sagemath-in-ubuntu-in-24-04>
200. Regina - Software for low-dimensional topology <https://regina-normal.github.io/>
201. Debian - USTC Mirror Help <https://mirrors.ustc.edu.cn/help/debian.html>
202. maxima package versions - Repology <https://repology.org/project/maxima/versions>
203. pull access denied repository does not exist or may require docker ... <https://stackoverflow.com/questions/48719921/pull-access-denied-repository-does-not-exist-or-may-require-docker-login>
204. 清华大学开源软件镜像站 | Tsinghua Open Source Mirror <https://mirrors.tuna.tsinghua.edu.cn/>
205. PractRand-fork/tools/RNG_test.cpp at master · GitHub https://github.com/rkern/PractRand-fork/blob/master/tools/RNG_test.cpp
206. How to tell if a Linux system is big endian or little endian? <https://serverfault.com/questions/163487/how-to-tell-if-a-linux-system-is-big-endian-or-little-endian>
207. EndianIssues - Linux Kernel Newbies <https://kernelnewbies.org/EndianIssues>
208. random - dieharder - proper binary input file - Stack Overflow <https://stackoverflow.com/questions/69090465/dieharder-proper-binary-input-file>
209. Debian (non-pure) 12 to 13 - Reddit https://www.reddit.com/r/debian/comments/1lihaup/debian_nonpure_12_to_13/
210. Release Notes for Debian 13 (trixie) <https://www.debian.org/releases/stable/release-notes/>
211. Fixed Debian 13 package installation issue - Facebook <https://www.facebook.com/groups/lifewithdebian/posts/10161667603158977/>
212. What's New in Debian 13: Full Update Review - Serverspace.io <https://serverspace.io/about/blog/whats-new-in-debian-13-full-update-review/>
213. Gitlab apt broken after debian upgrade to trixie <https://forum.gitlab.com/t/gitlab-apt-broken-after-debian-upgrade-to-trixie/129673>

214. Debian 13 "Trixie" Released: What's New in the Latest Version? <https://itsfoss.com/news/debian-13-release/>
215. Debian 13 trixie upgrade - GitHub Gist https://gist.github.com/yorickdowne/3cecc7b424ce241b173510e36754af47?permalink_comment_id=5696220
216. paranoid_crypto/docs/randomness_tests.md at main - GitHub https://github.com/google/paranoid_crypto/blob/main/docs/randomness_tests.md
217. GitHub - terrillmoore/NIST-Statistical-Test-Suite <https://github.com/terrillmoore/NIST-Statistical-Test-Suite>
218. NIST's statistical test suite for Random Number Generator ... - GitHub <https://github.com/GINARTeam/NIST-statistical-test>
219. [PDF] On the revision of NIST 800-22 Test Suites - Cryptology ePrint Archive <https://eprint.iacr.org/2022/540.pdf>
220. greendow/A-variant-of-NIST-SP-800-22-test-suite - GitHub <https://github.com/greendow/A-variant-of-NIST-SP-800-22-test-suite>
221. [PDF] A Statistical Test Suite for Random and Pseudorandom Number ... <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-22r1a.pdf>
222. [PDF] Applying the NIST Statistical Test Suite to a TRNG of an Open ... https://addi.ehu.es/bitstream/handle/10810/73912/preprint_applying_nist_statistical_iecas2025.pdf?sequence=3&isAllowed=y
223. How to use NIST SP 800-22 to check randomness of 128 bits output ... https://www.reddit.com/r/cryptography/comments/132zckc/how_to_use_nist_sp_80022_to_check_randomness_of/
224. DIEHARD and NIST-STS test results. (a) DIEHARD test results of four... https://www.researchgate.net/figure/DIEHARD-and-NIST-STS-test-results-a-DIEHARD-test-results-of-four-kinds-of-random_fig4_345820956
225. kravietz/nist-sts: NIST Statistical Test Suite (SP800-22rev1 ... - GitHub <https://github.com/kravietz/nist-sts>
226. [PDF] SMT solvers based Cryptanalysis of Block Ciphers - Web IITD Sites https://web.iitd.ac.in/~rksharma/Research%20Publications/Journal/Harish%20Rajesh%20Indivar%20RK%20Sharma_idea.pdf
227. SMT Solver-Based Cryptanalysis of Block Ciphers - Academia.edu https://www.academia.edu/89444833/SMT_Solver_Based_Cryptanalysis_of_Block_Ciphers
228. [PDF] ISeCure - The ISC International Journal of Information Security https://www.isecure-journal.com/article_242936_0ccd54a0135b72b482ed5d245b6180f2.pdf
229. [PDF] Comparative Analysis of SMT Solvers for Differential Cryptanalysis ... https://openaccess.city.ac.uk/id/eprint/35829/1/SMT_paper18.pdf

230. LLM2SMT: Building an SMT Solver with Zero Human-Written Code <https://arxiv.org/html/2603.06931v2>
231. Improving Bit-Blasting for Nonlinear Integer Constraints - 北京理工大学 <https://pure.bit.edu.cn/zh/publications/improving-bit-blasting-for-nonlinear-integer-constraints/>
232. Modelling finite field arithmetic mod p in Z_3 - Stack Overflow <https://stackoverflow.com/questions/71730323/modelling-finite-field-arithmetic-mod-p-in-z3>
233. [PDF] Satisfiability Modulo Finite Fields - Cryptology ePrint Archive <https://eprint.iacr.org/2023/091.pdf>
234. [2311.17896] On the geometry of tensor products over finite fields <https://arxiv.org/abs/2311.17896>
235. Tensorproduct of finite fields - Math Stack Exchange <https://math.stackexchange.com/questions/1526475/tensorproduct-of-finite-fields>
236. [PDF] finite field models in arithmetic combinatorics – ten years on <https://www.juliawolf.org/research/preprints/ffsurveyweb.pdf>
237. (PDF) New methods for finite field arithmetic / - ResearchGate https://www.researchgate.net/publication/34508216_New_methods_for_finite_field_arithmetic
238. https://scholar.google.com/citations?view_op=view_... https://scholar.google.com/citations?view_op=view_citation&hl=en&user=6MfCSXYAAAAJ&citation_for_view=6MfCSXYAAAAJ:SP6oXDckpogC
239. [PDF] Generating Optimized Sparse Matrix Vector Product over Finite Fields <https://www.lirmm.fr/~giorgi/icms2014-giovia.pdf>
240. Big-Endian Testing with QEMU - Hacker News <https://news.ycombinator.com/item?id=47626462>
241. C: Little Endian - Stack Overflow <https://stackoverflow.com/questions/35727864/c-little-endian>
242. Is there a system command, in Linux, that reports the endianness? <https://unix.stackexchange.com/questions/88934/is-there-a-system-command-in-linux-that-reports-the-endianness>
243. CI/testing for big endian architectures? - Julia Discourse <https://discourse.julialang.org/t/ci-testing-for-big-endian-architectures/26671>
244. [PDF] A Simple Deterministic Algorithm for Systems of Quadratic ... - HAL <https://hal.science/hal-03363031/document>
245. Arxiv-Social's post - Facebook https://www.facebook.com/arxivsocial/posts/arxivsocial_mathematicstile-on-a-noncommutative-iwasawa-main-conjecture-for-fun/505811843125879/

246. Key Recovery Attacks on ZIP Ciphers: Application to ZIP-AES and ... <https://tosc.iacr.org/index.php/ToSC/article/view/13008>
247. (PDF) Rigid Finite Simple Group Cryptography B; Tensor Product ... https://www.researchgate.net/publication/406931243_Rigid_Finite_Simple_Group_Cryptography_B_Tensor_Product_Cryptography-A_Secure_Framework_Based_on_the_Global_Sensitivity_of_Finite_Simple_Group_Representation_Categories
248. Differential Cryptanalysis in the Fixed-Key Model - YouTube <https://www.youtube.com/watch?v=qhraDM-daGU>
249. Cryptanalysis of an Image Encryption Algorithm Based on Two ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC10047706/>
250. Fukang Liu - Google Scholar <https://scholar.google.com.py/citations?user=DYdxh3gAAAAJ&hl=en>
251. [PDF] Triple Fixed-Point Theorems: Generalized Analysis with AI/Crypto ... <https://www.ejcam.com/index.php/ejcam/article/view/6609/2285>
252. How to read little endian integers in binary files in c++ - Stack Overflow <https://stackoverflow.com/questions/77456894/how-to-read-little-endian-integers-in-binary-files-in-c>
253. How important is it to check byte order when reading binary files? https://www.reddit.com/r/cpp_questions/comments/1ne2fh9/how_important_is_it_to_check_byte_order_when/
254. Byte Ordering for Integer Binary Data on Big Endian and Little ... http://documentation.sas.com/doc/en/pgmsascdc/v_068/leforinforref/p11zyf4zlzyjxen1irusv289xrrb.htm
255. NCL: Reading binary data https://www.ncl.ucar.edu/Applications/read_binary.shtml
256. Solving the Tensor Isomorphism Problem for special orbits with low ... <https://eprint.iacr.org/2024/337>
257. Fast and Robust Triple Tensor Decomposition With Data Corruption https://www.cmsworkshops.com/ICASSP2026/view_paper.php?PaperNum=10819&bare=1
258. [2307.02523] Fixed-point tensor is a four-point function - arXiv <https://arxiv.org/abs/2307.02523>
259. Xuejia Lai - Google Scholar <https://scholar.google.com/citations?user=B67-NyQAAAAJ&hl=en>
260. (2+1)d lattice models and tensor networks for gapped phases with ... <https://www.scipost.org/SciPostPhys.20.2.043>
261. How to Install Python 3.13 on Debian 13, 12 and 11 - LinuxCapable <https://linuxcapable.com/how-to-install-python-3-13-on-debian-linux/>

- 262. Debian 13 Trixie Released - DebianSupport.com <https://debiansupport.com/blog/debian-13-trixie-released/>
- 263. (PDF) Tensor products of unipotent characters of general linear ... https://www.researchgate.net/publication/223967000_Tensor_products_of_unipotent_characters_of_general_linear_groups_over_finite_fields
- 264. Irreducible tensor product modules over the Takiff Lie algebra ... - arXiv <https://arxiv.org/abs/2602.02021>
- 265. Tensor products of fields - abstract algebra - Math Stack Exchange <https://math.stackexchange.com/questions/292162/tensor-products-of-fields>
- 266. Tensor product of fields over integers - MathOverflow <https://mathoverflow.net/questions/231141/tensor-product-of-fields-over-integers>
- 267. [PDF] On deformations of module categories over finite tensor categories <https://d-nb.info/1316309398/34>
- 268. [PDF] SIMPLE TENSOR PRODUCTS Contents 1. Introduction 1 2. Finite ... <https://webusers.imj-prg.fr/~david.hernandez/Produit10.pdf>
- 269. Incorrect result of sum int-casted BitVec using Z3, Z3py <https://stackoverflow.com/questions/31876474/incorrect-result-of-sum-int-casted-bitvec-using-z3-z3py>
- 270. Vector finite fields of characteristic two as algebraic support of ... <https://www.math.md/en/publications/csjm/issues/v32-n1/13876/>
- 271. [PDF] Solving Problems with Magma <https://magma.maths.usyd.edu.au/magma/pdf/examples.pdf>
- 272. [PDF] Using Algebraic Geometry - eclass UoA <https://eclass.uoa.gr/modules/document/file.php/D231/Papers/Cox-UsingAlgebraicGeometry.pdf>
- 273. [PDF] Algebraic Geometry - James Milne <https://www.jmilne.org/math/CourseNotes/AG520.pdf>
- 274. (PDF) Gröbner Basis Representations of Sudoku - ResearchGate https://www.researchgate.net/publication/233652996_Grobner_Basis_Representations_of_Sudoku
- 275. Efficient modular arithmetic in Z3 - Stack Overflow <https://stackoverflow.com/questions/78144557/efficient-modular-arithmetic-in-z3>
- 276. Bitvectors | Online Z3 Guide <https://rlisahuang.com/z3guide/docs/theories/Bitvectors/>
- 277. [PDF] Compilation for Ragged Tensors with Minimal Padding - arXiv <https://arxiv.org/pdf/2110.10221>
- 278. Tensor fundamentals - Modular Documentation <https://docs.modular.com/max/develop/tensors/>

- 279. [PDF] A native tensor–vector multiplication algorithm for high performance ... <https://upcommons.upc.edu/bitstreams/ac844143-f99f-4722-945d-a4c20b63bb4c/download>
- 280. How to Write a Fast Matrix Multiplication from Scratch with Tensor ... <https://news.ycombinator.com/item?id=43736739>
- 281. Algorithm 862: MATLAB Tensor Classes for Fast Algorithm Prototyping <https://www.mathsci.ai/publication/bako06/>
- 282. Differential Cryptanalysis of the Reduced Pointer Authentication ... <https://eprint.iacr.org/2025/321>
- 283. https://scholar.google.com/citations?view_op=view_... https://scholar.google.com/citations?view_op=view_citation&hl=en&user=6_T-WhoAAAAJ&citation_for_view=6_T-WhoAAAAJ:Tyk-4Ss8FVUC
- 284. Similar articles - Journal of Zhejiang University <https://jzus.zju.edu.cn/article.php?doi=10.1631/FITEE.2300848>
- 285. [PDF] Cube-based cryptanalysis of Ascon and Gift - HAL <https://hal.science/hal-04692191v1/document>
- 286. 5. Issues to be aware of for trixie — release-notes documentation <https://www.debian.org/releases/trixie/release-notes/issues.html>
- 287. Debian -- Details of package python3 in trixie <https://packages.debian.org/trixie/python3>
- 288. Python 3.14 backport for Debian 13 trixie - GitHub <https://github.com/syssi/python3.14-backport/>
- 289. Debian 13 “Trixie” | Hacker News <https://news.ycombinator.com/item?id=44848782>
- 290. endian(3) - Linux manual page - man7.org <https://man7.org/linux/man-pages/man3/endian.3.html>
- 291. Building a CPRNG called Terninger - Part 4 Random Tests <https://blog.ligos.net/2017-06-02/Building-A-CRNG-Terninger-4-Random-Tests.html>
- 292. Is it possible that PracRand did not detect looping of the generator? <https://crypto.stackexchange.com/questions/98129/is-it-possible-that-pracrand-did-not-detect-looping-of-the-generator>
- 293. endian(3) - Linux man page <https://linux.die.net/man/3/endian>
- 294. Debian -- Packages <https://packages.debian.org/>
- 295. Cant install packages on Debian 13 kde - Reddit https://www.reddit.com/r/debian/comments/1noh8v2/cant_install_packages_on_debian_13_kde/
- 296. [Software] [Solved] trixie Release' does not have a Release file <https://forums.debian.net/viewtopic.php?t=163660>

297. [SOLVED] [HELP] Debian Linux 13.2 - apt does not work <https://www.linuxquestions.org/questions/linux-newbie-8/%5Bhelp%5D-debian-linux-13-2-apt-does-not-work-4175756687-print/>
298. Debian 13 Trixie complete sources.list - GitHub Gist <https://gist.github.com/shiwildy/47c5085ff04bc2cee01d28f4151e96a3>
299. MaXX Interactive Desktop is the little brother of the great SGI ... <https://www.facebook.com/groups/retrocomputers/posts/6516103935086099/>
300. [PDF] Sage 教程 [http://www.sandal.tw/upload/SageTutorial\(%E4%B8%AD%E6%96%87\).pdf](http://www.sandal.tw/upload/SageTutorial(%E4%B8%AD%E6%96%87).pdf)
301. [PDF] HANDBOOK OF MAGMA FUNCTIONS <https://www.math.uzh.ch/sepp/magma-2.19.8-cr/HandbookVolume10.pdf>
302. Debian -- Software Packages in "trixie" <https://packages.debian.org/stable/allpackages>
303. Debian -- Software Packages in "forky" <https://packages.debian.org/testing/allpackages>
304. <https://packages.debian.org/trixie/amd64/allpackag...> <https://packages.debian.org/trixie/amd64/allpackages?format=txt.gz>
305. Debian -- Software Packages in "trixie", Subsection devel <https://packages.debian.org/stable/devel/>
306. <https://packages.debian.org/source/trixie/allpacka...> <https://packages.debian.org/source/trixie/allpackages?format=txt.gz>
307. Paquets logiciels dans « trixie - Debian -- Packages <https://packages.debian.org/fr/stable/allpackages>
308. compact compressed textlist - Debian -- Packages <https://packages.debian.org/testing/allpackages?format=txt.gz>
309. python-debian - Debian Package Tracker <https://tracker.debian.org/python-debian>
310. Debian Contributions: Tracker.debian.org updates, Salsa CI ... <https://www.freexian.com/blog/debian-contributions-12-2024/>
311. [PDF] an introduction to block cipher cryptanalysis <https://open.metu.edu.tr/bitstream/handle/11511/118380/H%C3%BCseyin%20%C5%9Eencan.pdf>
312. Questions on multiplication over $GF(2^{32})$ when implementing ... <https://crypto.stackexchange.com/questions/114075/questions-on-multiplication-over-gf232-when-implementing-monoid-crc32>
313. [PDF] PART 4: Finite Fields of the Form $GF(2^n)$ Theoretical Underpinnings ... <https://engineering.purdue.edu/kak/compsec/NewLectures/Lecture7.pdf>
314. [PDF] Efficient and Low-complexity Hardware Architecture of Gaussian ... <https://eprint.iacr.org/2015/1109.pdf>

315. [PDF] Efficient Software Implementations of Large Finite Fields $GF(2^n)$ for ... <https://lihaoxu.eng.wayne.edu/NISL/Papers/Journals/tos-gf.pdf>
316. [PDF] Efficient Software Implementation for Finite Field Multiplication in ... <https://www.princeton.edu/~rblee/ELE572Papers/Fall04Readings/NingYin-FiniteFieldMul.pdf>
317. How could one implement multiplication in finite fields? <https://stackoverflow.com/questions/1935658/how-could-one-implement-multiplication-in-finite-fields>
318. [PDF] A New Architecture for a Parallel Finite Field Multiplier with Low ... <https://my.eng.utah.edu/~kalla/Comp-Algebra/paar-tcomp-1996-composite-field-mult.pdf>
319. Complexity of Low-Degree Skew Polynomial Multiplication over ... <https://arxiv.org/html/2607.00476v1>
320. Low-complexity bit-parallel canonical and normal basis multipliers ... <https://www.semanticscholar.org/paper/68961dd563044fe2f3a14fffc0af71af393ef1c9>
321. [PDF] David A. Cox John Little Donal O'Shea http://ndl.ethernet.edu.et/bitstream/123456789/22981/1/David%20A.%20Cox_.pdf
322. PractRand/doc/Tests_overview.txt at master · GitHub https://github.com/MartyMacGyver/PractRand/blob/master/doc/Tests_overview.txt
323. How to test STDIN read error - linux - Stack Overflow <https://stackoverflow.com/questions/22439841/how-to-test-stdin-read-error>
324. Debian -- Software Packages in "trixie", Subsection python <https://packages.debian.org/trixie/python/>
325. gmpy2/gmpy2: General Multi-Precision arithmetic for Python (GMP ... <https://github.com/aleaxit/gmpy>
326. Gmpy2: Python Package Guide 2026 - Generalist Programmer <https://generalistprogrammer.com/tutorials/gmpy2-python-package-guide>
327. sage/pyproject.toml at develop · sagemath/sage - GitHub <https://github.com/sagemath/sage/blob/develop/pyproject.toml>
328. PEP 757 – C API to import-export Python integers <https://peps.python.org/pep-0757/>
329. Python 3.12 · Issue #9197 · numba/numba - GitHub <https://github.com/numba/numba/issues/9197>
330. Review python versions for 1.14 · Issue #27341 · sympy ... - GitHub <https://github.com/sympy/sympy/issues/27341>
331. z3-solver 4.13.1.0 - PyPI <https://pypi.org/project/z3-solver/4.13.1.0/>
332. What methods does Z3 use to solve quantifier-free bit-vector ... <https://stackoverflow.com/questions/7268221/what-methods-does-z3-use-to-solve-quantifier-free-bit-vector-formulas-qf-bv>

- 333. Debian -- Details of source package z3 in trixie <https://packages.debian.org/stable/source/z3>
- 334. Debian -- Details of package libz3-4 in trixie <https://packages.debian.org/trixie/libz3-4>
- 335. gmpy2 - PyPI <https://pypi.org/project/gmpy2/>
- 336. gmpy2 · Anitya - Release-Monitoring.org <https://release-monitoring.org/project/python-gmpy2>
- 337. Installation — gmpy2 2.4.0a1.dev20+g33e027b46 documentation <https://gmpy2.readthedocs.io/en/latest/install.html>
- 338. Releases · gmpy2/gmpy2 - GitHub <https://github.com/aleaxit/gmpy/releases>